Laboratory 7
Variant 1
Group 16

By Anatoli Abahumna and Leonardo Meloni

1 Introduction

Variant 1 requires simple addition and subtraction of dates by a number of days. The input date is written as a string in the `"DDMM"` format, for example `"2205"` for 22 May. The year is fixed to 2024, which is a leap year with 366 days. The offset `N` is assumed to be an integer in the range from 0 to 366.

The solution treats every valid date as an ordinal day of the year. A date is converted from `(Day, Month)` into an ordinal value between 1 and 366. Addition and subtraction are then normal integer operations. The resulting ordinal value is accepted only if it still belongs to 2024, and is then converted back to the `"DDMM"` format. Dates that would leave 2024 fail instead of wrapping to another year.

Prolog is suitable for this task because the program can be expressed as facts and rules. Facts describe the calendar, while rules describe validation, conversion, and date arithmetic. Important Prolog features used here are unification, recursive rules, pattern matching through predicate heads, and backtracking. The main advantages are concise rule-based code and direct expression of the calendar logic. The disadvantages are that formatted input and output can be verbose, and recursive control flow may be less transparent for beginners.

2 Implementation

The program starts with month length facts for the leap year 2024. This keeps the calendar data in one place.

```
month_days(1, 31).  month_days(2, 29).  month_days(3, 31).
month_days(4, 30).  month_days(5, 31).  month_days(6, 30).
month_days(7, 31).  month_days(8, 31).  month_days(9, 30).
month_days(10, 31). month_days(11, 30). month_days(12, 31).
```

The input string is split into four characters. The first two characters become the day, and the last two become the month. The reverse operation uses `pad2/2` so that dates such as 5 June are printed as `"0506"` rather than `"56"`.

```
parse_date(Str, D, M) :-
    string_chars(Str, [D1, D2, M1, M2]),
    number_chars(D, [D1, D2]),
    number_chars(M, [M1, M2]).

pad2(N, S) :-
    N < 10,
    !,
    format(string(S), "0~w", [N]).
pad2(N, S) :-
```

```
    format(string(S), "~w", [N]).

make_date_string(D, M, Str) :-
    pad2(D, DS),
    pad2(M, MS),
    string_concat(DS, MS, Str).
```

Date validation checks that the month is in the calendar, that the day is positive, and that the day does not exceed the length of that month. The offset is also checked against the assignment range.

```
valid_date(D, M) :-
    integer(M), M >= 1, M =< 12,
    integer(D), D >= 1,
    month_days(M, MD), D =< MD.

valid_offset(N) :-
    integer(N), N >= 0, N =< 366.
```

The central idea is the bijection between a calendar date and an ordinal day. The predicate `date_to_ordinal/3` adds the lengths of all previous months to the current day. The predicate `ordinal_to_date/3` reverses that process by subtracting whole months until the remaining value fits into the current month.

```
date_to_ordinal(D, 1, D).
date_to_ordinal(D, M, Ord) :-
    M > 1,
    PrevM is M - 1,
    month_days(PrevM, PD),
    date_to_ordinal(D, PrevM, PrevOrd),
    Ord is PrevOrd + PD.

ordinal_to_date(Ord, D, M) :-
    integer(Ord),
    ordinal_to_date_(Ord, 1, D, M).

ordinal_to_date_(Ord, M, Ord, M) :-
    M =< 12,
    month_days(M, MD),
    Ord >= 1,
    Ord =< MD,
    !.
ordinal_to_date_(Ord, M, D, OutM) :-
    M < 12,
    month_days(M, MD),
    Ord > MD,
    NextOrd is Ord - MD,
    NextM is M + 1,
    ordinal_to_date_(NextOrd, NextM, D, OutM).
```

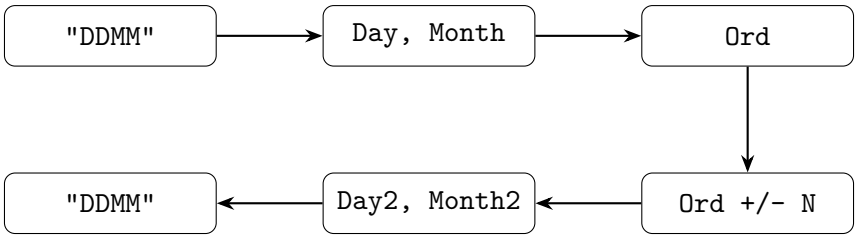


Figure 1: Data flow used for date addition and subtraction.

The public predicates combine validation, ordinal conversion, arithmetic, range checking, and formatting. The three-argument versions are useful for testing because they bind the result. The two-argument versions match the query style from the assignment and print the result.

```
add_date(DateStr, N, Result) :-
    valid_offset(N),
    parse_date(DateStr, D, M),
    valid_date(D, M),
    date_to_ordinal(D, M, Ord),
    NewOrd is Ord + N,
    NewOrd >= 1,
    NewOrd =< 366,
    ordinal_to_date(NewOrd, D2, M2),
    make_date_string(D2, M2, Result).

sub_date(DateStr, N, Result) :-
    valid_offset(N),
    parse_date(DateStr, D, M),
    valid_date(D, M),
    date_to_ordinal(D, M, Ord),
    NewOrd is Ord - N,
    NewOrd >= 1,
    NewOrd =< 366,
    ordinal_to_date(NewOrd, D2, M2),
    make_date_string(D2, M2, Result).

add_date(DateStr, N) :-
    add_date(DateStr, N, R),
    format("~q~n", [R]).

sub_date(DateStr, N) :-
    sub_date(DateStr, N, R),
    format("~q~n", [R]).
```

3 Discussion

The following test cases cover the examples from the task, leap-year behavior, and boundary cases at the start and end of the year.

Query	Expected result	Reason
`add_date("2205", 14)`	`"0506"`	example from the task
`sub_date("2205", 10)`	`"1205"`	example from the task
`add_date("2802", 1)`	`"2902"`	lands on leap day
`add_date("2902", 1)`	`"0103"`	moves past leap day
`sub_date("0103", 1)`	`"2902"`	moves back to leap day
`add_date("3112", 1)`	fails	would leave 2024
`sub_date("0101", 1)`	fails	would leave 2024

Failing outside the range of 2024 is the correct behavior because the task fixes the year to 2024. Wrapping from 31 December to 1 January or from 1 January to 31 December would silently change the meaning of the input data and introduce an extra year assumption that is not part of the variant.

4 Conclusion

The solution shows how a date arithmetic problem can be expressed in Prolog using facts, recursive rules, validation predicates, and formatted output. The ordinal-day technique keeps addition and subtraction simple while still handling the leap day correctly.

The main lesson is that declarative rules can describe both the calendar structure and the conversion process clearly. The main difficulty in this task is avoiding off-by-one errors around month boundaries, especially around 29 February and the first and last days of the year. A possible improvement would be to generalize the program to arbitrary years by adding a leap-year predicate and parameterizing month lengths by year.